

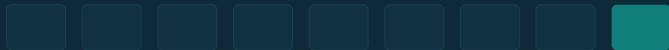


STREAM PROCESSING AVEC APACHE KAFKA

TP — Flux stateless

Router, éclater et filtrer des flux Kafka Streams

ESGI · 5^e année · Data Engineering · 2025–2026



SÉANCE 10

Objectifs du TP

- **Consolider le DSL stateless**

Composer `filter`, `mapValues`, `flatMapValues` dans une vraie topologie.

- **Router un flux avec `split()`**

Diriger les enregistrements vers plusieurs topics selon des prédicats.

- **Éclater et nettoyer**

Une commande → plusieurs articles ; isoler les messages invalides.

- **Observer le parallélisme**

Plusieurs instances, plusieurs partitions, une seule `application.id`.

SÉANCE 10

Le scénario



- **Entrée** `orders` : lignes `orderId; amount; currency; sku1 | sku2 | ...`
- **Sorties** : `orders-priority` et `orders-standard` (par montant), `order-items` (un article par ligne), `orders-rejected` (lignes malformées).

01

- PARTIE 1

Les patterns stateless

La boîte à outils stateless

Opérateur	Effet	Clé préservée
<code>filter</code> <code>filterNot</code>	/ garder / exclure selon un prédicat	oui
<code>mapValues</code>	transformer la valeur	oui
<code>flatMapValues</code>	une valeur → plusieurs valeurs	oui
<code>selectKey / map</code>	changer la clé	non (repartition)
<code>split()</code>	router en branches	oui
<code>merge</code>	fusionner deux flux	oui

Tout ce qui touche à la **clé** peut déclencher une *repartition* (un topic interne). À éviter sans raison.

Diriger un flux avec `split()`

Router par palier de montant vers deux topics

```
valid.split(Named.as("tier-"))  
  .branch((k, v) -> amountOf(v) >= 100.0,  
    Branched.withConsumer(ks -> ks.to("orders-priority")))  
  .defaultBranch(  
    Branched.withConsumer(ks -> ks.to("orders-standard")));
```

`split()` remplace l'ancien `branch()` (retiré en 4.0). Chaque branche reçoit un `Branched`.

flatMapValues et dead-letter

Isoler les invalides, puis éclater en articles

```
// Lignes malformees -> topic de rejet (dead-letter)
orders.filterNot(App::isValid).to("orders-rejected");

// Garder les valides, puis une ligne par article
KStream<String, String> valid = orders.filter(App::isValid);
valid.flatMapValues(App::explodeItems).to("order-items");
```

`filterNot` sépare les rejets ; `flatMapValues` émet 0, 1 ou N valeurs en gardant la clé.

02

- PARTIE 2

Le TP — pas à pas

Cluster et topics

Demarrer le cluster et creer les cinq topics

```
docker compose up -d          # 3 brokers KRaft

for t in orders orders-rejected orders-priority \
        orders-standard order-items; do
    kafka-topics.sh --bootstrap-server localhost:29092 \
        --create --topic $t --partitions 3 --replication-factor 3
done
```

■ Projet

Maven + kafka-streams (sur Maven Central).

■ Entrée

Une commande par ligne ; articles séparés par |.

TP · ÉTAPES

Construire la topologie

- 1 **Valider** : `filterNot (isValid)` → `orders-rejected` ; garder le flux valid.
- 2 **Router** : `split ()` sur le montant → `orders-priority` / `orders-standard`.
- 3 **Éclater** : `flatMapValues (explodeItems)` → `order-items`.
- 4 Lancer l'application, produire des commandes (dont une malformée).
- 5 Vérifier *chaque* topic de sortie, puis lancer une **deuxième instance**.

Vérifier les sorties et l'échelle

Lire une sortie, puis inspecter le groupe

```
# Une commande >= 100 doit apparaitre ici
kafka-console-consumer.sh --bootstrap-server localhost:29092 \
  --topic orders-priority --from-beginning --max-messages 5
# La repartition entre instances (le groupe = application.id)
kafka-consumer-groups.sh --bootstrap-server localhost:29092 \
  --describe --group tp10-order-router
```

■ Quatre sorties

Chaque topic reçoit exactement ce que sa branche décrit.

■ Réparti

Les partitions se distribuent entre les instances actives.

Pour aller plus loin

Objectif — enrichir la topologie et mesurer le cout d'une reclé.

■ 1. Recléer

`selectKey` sur la devise, puis repérer le topic de `repartition`.

■ 2. Fusionner

Recombinaison de deux branches avec `merge` et écrire un flux unifié.

■ 3. Mesurer

`peek` pour compter les enregistrements par branche.

OUVERTURE

La limite du stateless

- Tout ce qu'on a fait traite chaque enregistrement **isolément** : aucune mémoire d'un message à l'autre.
- Dès qu'il faut **compter**, **cumuler**, **joindre** ou **fenêtrer**, il faut de l'**état** — un *state store* sauvegardé dans un changelog.

■ Acquis

Filtrer, transformer, router, éclater : un ETL de streaming.

À venir

`groupByKey`, `count`,
`aggregate` et les *state stores*.

SÉANCE 10

À retenir

- ✓ Une topologie peut se **ramifier** : valider, router, éclater dans un même graphe.
- ✓ `split()` route en branches (l'ancien `branch()` a disparu en 4.0) ; `defaultBranch` attrape le reste.
- ✓ `filterNot` vers un topic de rejet = un **dead-letter** sans perte de données.
- ✓ `flatMapValues` éclate une valeur en plusieurs, en **préservant la clé**.
- ✓ Changer la clé (`selectKey`, `map`) coûte une **repartition** : à n'utiliser qu'au besoin.



LA SUITE — SÉANCE 11

Traitement avec état

Compter, cumuler et agréger des flux par clé, avec des state stores sauvegardés dans des changelogs.